boba_extension_dim07

Dimension 07: Torrent File Detection, Download & Bencode Parsing

Research Summary

This document provides a comprehensive technical reference for implementing torrent file detection, download, bencode parsing, infohash extraction, and magnet URI generation in a browser extension context. It covers the BitTorrent protocol specifications (BEP 3, BEP 52), bencode encoding/decoding, CORS handling strategies for cross-origin torrent downloads, and complete code implementations using browser-compatible JavaScript/TypeScript.

Table of Contents

---

1. BitTorrent File Format Specification

1.1 BEP 3: The BitTorrent Protocol Specification

The BitTorrent protocol specification is defined in BEP 3 (BitTorrent Enhancement Proposal 3), authored by Bram Cohen.

Claim: BEP 3 defines the metainfo file (.torrent) as a bencoded dictionary containing an `announce` URL and an `info` dictionary. **Source:** BitTorrent.org (Official BEP 3) **URL:** https://www.bittorrent.org/beps/bep_0003.html **Date:** 2008-01-10 (Final status) **Excerpt:** > "Metainfo files (also known as .torrent files) are bencoded dictionaries with the following keys: `announce` — The URL of the tracker. `info` — This maps to a dictionary, with keys described below." **Context:** This is the authoritative specification for the original BitTorrent protocol. **Confidence:** high

1.2 Metainfo File Structure

A .torrent file is a bencoded dictionary with the following keys:

Required Keys

Key	Type	Description
`announce`	string	The URL of the tracker
`info`	dictionary	A dictionary that describes the file(s) being shared

Optional Keys

Key	Type	Description
`announce-list`	list	List of lists of tracker URLs (tiered trackers, BEP 12)
`creation date`	integer	Creation time in Unix epoch format
`comment`	string	Free-form comment from the author

Key	Type	Description
`created by`	string	Name and version of the program used to create the .torrent
`encoding`	string	Encoding of strings in the info dictionary (e.g., UTF-8)

Source: https://www.bittorrent.org/beps/bep_0003.html, https://wiki.theory.org/BitTorrentSpecification **Confidence:** high

1.3 The Info Dictionary

The `info` dictionary is the most critical part of a .torrent file. It contains:

Common Keys (always present)

Key	Type	Description
`piece length`	integer	Number of bytes in each piece (typically power of 2: 256KB = 2^18, 1MB = 2^20, 4MB = 2^22)
`pieces`	string	Concatenation of all 20-byte SHA-1 hash values, one per piece. Length = multiple of 20
`name`	string	Suggested filename (single-file) or directory name (multi-file)
`private`	integer	If set to 1, the client MUST disable DHT and PEX (BEP 27)

Single-File Mode Keys

Key	Type	Description
`length`	integer	Length of the file in bytes

Multi-File Mode Keys

Key	Type	Description
`files`	list	List of dictionaries, one per file

Each file dictionary in the `files` list contains:

Key	Type	Description
`length`	integer	Length of the file in bytes
`path`	list	List of strings representing the relative path (directories + filename)

Source: https://www.bittorrent.org/beps/bep_0003.html **Excerpt:** > “The `name` key maps to a string which is the suggested name to save the file (or directory) as. It is purely advisory. `piece length` maps to the number of bytes in each piece the file is split into. `pieces` maps to a string whose length is a multiple of 20. It is to be subdivided into strings of length 20, each of which is the SHA1 hash of the piece.” **Confidence:** high

1.4 BitTorrent v2 (BEP 52)

BitTorrent v2 uses SHA-256 instead of SHA-1 and introduces several changes:

Claim: BitTorrent v2 infohash uses SHA-256 instead of SHA-1, producing a 32-byte hash. **Source:** BEP 52 (BitTorrent Protocol Specification v2) **URL:** https://www.bittorrent.org/beps/bep_0052.html **Excerpt:** > “For `meta version` 2 SHA2-256 is used. The info-hash must be the hash of the encoded form as found in the .torrent file.” **Context:** BEP 52 is currently in Draft status. In practice, most torrents are still v1. **Confidence:** high

Key differences in v2: - Infohash: SHA-256 (32 bytes) instead of SHA-1 (20 bytes) - Piece hashes: SHA-256 (32 bytes each) instead of SHA-1 (20 bytes each) - File tree structure: Uses a merkle tree per file instead of flat piece hashes - Magnet links use `urn:btmh:` prefix for v2 hashes (with `0x12 0x20` multihash prefix) - Hybrid torrents contain both v1 and v2 data for backwards compatibility

Claim: For v2 torrents used with DHT/trackers, the SHA-256 infohash is truncated to 20 bytes for compatibility. **Source:** libtorrent blog post on BitTorrent v2 **URL:** https://blog.libtorrent.org/2020/09/bittorrent-v2/ **Excerpt:** > “The info-dictionary is also computed by SHA-256, which poses a compatibility challenge with the DHT and trackers, which have protocols that expect 20 byte hashes. To handle this, DHT- and tracker announces and lookups for v2 torrents use the SHA-256 info-hash truncated to 20 bytes.” **Confidence:** high

---

2. Bencode Encoding Specification

2.1 Overview

Bencode (pronounced “B-encode”) is the serialization format used by BitTorrent. It supports four data types: byte strings, integers, lists, and dictionaries.

Source: BEP 3 / Multiple authoritative references **Confidence:** high

2.2 Encoding Rules

Byte Strings

Format: `<length in base 10 ASCII>:<string contents>`

- The length is encoded in base 10, followed by a colon
- The string can contain any binary data (including null bytes)
- No escaping is needed
- Zero-length strings are valid: `0:`

Examples: | Encoded | Decoded | |———|———| | `4:spam` | `"spam"` | | `0:` | `""` (empty string) | | `10:helicopter` | `"helicopter"` |

Excerpt (from Debian BitTorrent protocol documentation): > “Strings are length-prefixed base ten followed by a colon and the string. For example `4:spam` corresponds to ‘spam’.” [267]

Integers

Format: `i<integer in base 10 ASCII>e`

- Wrapped with `i` prefix and `e` suffix
- No size limitation (but signed 64-bit handling is recommended)
- No leading zeros allowed (except `0` itself)
- Negative values use `-` prefix
- `i-0e` is invalid

Examples: | Encoded | Decoded | |———|———| | `i3e` | `3` | | `i-3e` | `-3` | | `i0e` | `0` |

Excerpt: > “Integers are represented by an ‘i’ followed by the number in base 10 followed by an ‘e’. For example `i3e` corresponds to 3 and `i-3e` corresponds to -3. Integers have no size limitation. `i-0e` is invalid. All encodings with a leading zero, such as `i03e`, are invalid, other than `i0e`.” [267]

Lists

Format: `l<bencoded values>e`

- Wrapped with `l` prefix and `e` suffix
- Elements are bencoded values in order
- Can contain any bencode type including other lists and dictionaries

Examples: | Encoded | Decoded | |———|———| | `l4:spam4:eggse` | `["spam", "eggs"]` | | `le` | `[]` (empty list) | | `l4:spami42ee` | `["spam", 42]` |

Dictionaries

Format: `d<bencoded string><bencoded value>...e`

- Wrapped with `d` prefix and `e` suffix
- Keys must be bencoded strings
- Keys must appear in **lexicographically sorted order** (sorted as raw byte strings, not alphanumerically)
- Values can be any bencode type

Examples: | Encoded | Decoded | |———|———| | `d3:cow3:moo4:spam4:eggse` | `{"cow": "moo", "spam": "eggs"}` | | `d4:spaml1:a1:bee` | `{"spam": ["a", "b"]}` | | `de` | `{}` (empty dict) |

Critical note on key ordering: > “Keys must be strings and appear in sorted order (sorted as raw strings, not alphanumerics).” [267]

This ordering requirement is essential for infohash computation — the bencoded info dictionary must have its keys in sorted order for the hash to be deterministic.

2.3 Bencode Decoder Decision Tree

To parse bencode, read the first character and decide:

First character	Type	Action
`i`	Integer	Read until `e` delimiter
`l`	List	Recursively parse elements until `e`
`d`	Dictionary	Recursively parse key-value pairs until `e`
`0-9`	Byte string	Read length, colon, then string contents
`e`	End marker	Return from current list/dict

3. Detecting .torrent Links on Web Pages

3.1 Link Pattern Detection

Torrent links can be detected through multiple strategies:

Strategy 1: HREF pattern matching (`.torrent` files)

```
// Regex to detect .torrent file links
const TORRENT_LINK_REGEX = /\.torrent(?:\?.*)?(?:#.*)?$/i;

// More comprehensive pattern that handles query parameters
const TORRENT_HREF_PATTERN = /(?:^|[^\w])(https?:\/\/[^\s\"<>]+
        \.torrent(?:\?[^\s\"<>]*)?)/gi;

// Check if an anchor element is a torrent link
function isTorrentLink(element: HTMLAnchorElement): boolean {
  const href = element.href || '';
  return TORRENT_LINK_REGEX.test(href);
}
```

Strategy 2: Magnet URI detection

```
// Magnet URI pattern
const MAGNET_URI_REGEX = /^magnet:\?xt=urn:btih:[a-f0-9]{40}/i;

// More flexible pattern
const MAGNET_LINK_PATTERN = /magnet:\?xt=urn:btih:[a-f0-9]{40}
        [^\s\"<>]*/gi;

// Check for magnet links
function isMagnetLink(element: HTMLAnchorElement): boolean {
  const href = element.href || '';
```

```
  return href.startsWith('magnet:?');
}
```

Strategy 3: Content-Type detection (for dynamically loaded content)

```
// Check the type attribute or download attribute
function hasTorrentAttributes(element: HTMLAnchorElement):
        boolean {
  const download = element.getAttribute('download') || '';
  const type = element.getAttribute('type') || '';
  return download.toLowerCase().endsWith('.torrent') ||
         type === 'application/x-bittorrent';
}
```

3.2 Complete DOM Scanning Strategy

```
interface TorrentLink {
  url: string;
  type: 'torrent-file' | 'magnet';
  element?: HTMLAnchorElement;
  text?: string;
}

/**
 * Scan a DOM element for all torrent-related links
 */
function scanForTorrentLinks(container: HTMLElement =
        document.body): TorrentLink[] {
  const links: TorrentLink[] = [];
  const anchors = container.querySelectorAll('a[href]');

  for (const anchor of anchors) {
    const el = anchor as HTMLAnchorElement;
    const href = el.href || '';

    if (href.startsWith('magnet:?')) {
      links.push({
        url: href,
        type: 'magnet',
        element: el,
        text: el.textContent || undefined
      });
    } else if (TORRENT_LINK_REGEX.test(href)) {
      links.push({
        url: href,
        type: 'torrent-file',
        element: el,
        text: el.textContent || undefined
      });
    }
```

```
  }

  return links;
}
```

3.3 Common Torrent Site URL Patterns

```
// Common patterns seen on torrent sites
const COMMON_PATTERNS = {
  // Direct .torrent links
  directTorrent: /\.torrent$/i,

  // Magnet links with various xt formats
  magnetStandard: /^magnet:\?xt=urn:btih:[a-f0-9]{40}/i,
  magnetBase32: /^magnet:\?xt=urn:btih:[a-z2-7]{32}/
        i,  // 32-char base32

  // Download pages that redirect to .torrent
  downloadPage: /\/download\/\?.*torrent/i,

  // Tracker-specific patterns
  passkeyPattern: /\/\w{32}\/\w+\.torrent$/,  // /PASSKEY/
        filename.torrent
};
```

---

4. Downloading .torrent Files (CORS Handling)

4.1 The CORS Problem

Claim: Chrome Extension content scripts CANNOT make cross-origin `fetch()` requests starting from Chrome 85. They must delegate to the background service worker. **Source:** Chromium Security Documentation **URL:** https://www.chromium.org/Home/chromium-security/extension-content-script-fetches/ **Date:** 2020-09-17 **Excerpt:** > “Content scripts will lose the ability to fetch cross-origin data from origins in their extension’s permissions, and they will only be able to fetch data that the underlying page itself has access to. To fetch additional data, content scripts can send messages to their extension’s background pages, which can relay data from sources that the extension author expects.” **Context:** This is a critical architecture constraint for browser extensions that download torrent files from arbitrary URLs. **Confidence:** high

4.2 Architecture: Content Script -> Service Worker -> fetch()

```
Content Script                          Service Worker (MV3)
------------                            --------------------
1. Detect torrent link
2. Send message to SW    --------->  3. Receive message
                                     4. fetch() torrent file
                                     5. Parse/Process
6. Receive response      <---------  6. Send response back
```

4.3 Manifest Configuration

```
{
  "manifest_version": 3,
  "name": "Torrent Detector Extension",
  "permissions": ["activeTab"],
  "host_permissions": [
    "https://*/*",
    "http://*/*"
  ],
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "content_scripts": [
    {
      "matches": ["https://*/*", "http://*/*"],
      "js": ["content.js"]
    }
  ]
}
```

Important: `host_permissions` with `<all_urls>` or `https://*/*` is required for the service worker to fetch cross-origin resources. The user will be prompted to approve this permission on install.

4.4 Service Worker: Cross-Origin fetch Implementation

```
// background.ts (Service Worker)

interface TorrentDownloadRequest {
  action: 'downloadTorrent';
  url: string;
  timeout?: number;
}

interface TorrentDownloadResponse {
  success: boolean;
```

```
  data?: ArrayBuffer;
  infoHash?: string;
  magnetUri?: string;
  metadata?: TorrentMetadata;
  error?: string;
}

interface TorrentMetadata {
  name: string;
  announce: string[];
  pieceLength: number;
  totalSize: number;
  files: { path: string; length: number }[];
  isPrivate: boolean;
}

chrome.runtime.onMessage.addListener(
  (
    request: TorrentDownloadRequest,
    sender: chrome.runtime.MessageSender,
    sendResponse: (response: TorrentDownloadResponse) => void
  ): boolean => {
    if (request.action === 'downloadTorrent') {
      downloadAndParseTorrent(request.url, request.timeout)
        .then(sendResponse)
        .catch((error) => {
          sendResponse({
            success: false,
            error: error instanceof Error ? error.message :
        String(error)
          });
        });
      return true; // Will respond asynchronously
    }
    return false;
  }
);

async function downloadAndParseTorrent(
  url: string,
  timeout: number = 30000
): Promise<TorrentDownloadResponse> {
  try {
    // Fetch the .torrent file through the service worker
    // This bypasses CORS restrictions that would apply in content
        scripts
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(),
        timeout);

    const response = await fetch(url, {
      method: 'GET',
```

```
      signal: controller.signal,
      headers: {
        'Accept': 'application/x-bittorrent,*/*',
      }
    });
    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: $
        {response.statusText}`);
    }

    // Read the response as ArrayBuffer
    const arrayBuffer = await response.arrayBuffer();

    // Parse the torrent file
    const result = parseTorrentFile(new Uint8Array(arrayBuffer));

    return {
      success: true,
      data: arrayBuffer,
      infoHash: result.infoHash,
      magnetUri: result.magnetUri,
      metadata: result.metadata
    };

  } catch (error) {
    return {
      success: false,
      error: error instanceof Error ? error.message :
        String(error)
    };
  }
}
```

4.5 Content Script: Requesting Torrent Downloads

```
// content.ts (Content Script)

async function fetchTorrentFromBackground(url: string):
        Promise<TorrentDownloadResponse> {
  return new Promise((resolve, reject) => {
    chrome.runtime.sendMessage(
      { action: 'downloadTorrent', url, timeout: 30000 },
      (response) => {
        if (chrome.runtime.lastError) {
          reject(new Error(chrome.runtime.lastError.message));
        } else {
          resolve(response);
        }
      }
```

```
    );
  });
}

// Example: Download all torrent links found on the page
async function processTorrentLinks(): Promise<void> {
  const links = scanForTorrentLinks();

  for (const link of links.filter(l => l.type === 'torrent-
        file')) {
    try {
      const result = await fetchTorrentFromBackground(link.url);
      if (result.success) {
        console.log(`Torrent: ${result.metadata?.name}, InfoHash:
        ${result.infoHash}`);
      } else {
        console.error(`Failed to download ${link.url}: $
        {result.error}`);
      }
    } catch (error) {
      console.error(`Error downloading ${link.url}:`, error);
    }
  }
}
```

4.6 Alternative: Using `no-cors` Mode (Limited)

```
// NOTE: no-cors mode returns an opaque response that cannot be
        read
// This is generally NOT useful for .torrent files since you need
        the content
// However, it can be used for HEAD requests to check existence

async function checkTorrentExists(url: string): Promise<boolean> {
  try {
    const response = await fetch(url, {
      method: 'HEAD',
      mode: 'no-cors' // Returns opaque response
    });
    // With no-cors, response.ok will be false but response.type
        === 'opaque'
    return response.type === 'opaque' || response.ok;
  } catch {
    return false;
  }
}
```

---

5. Bencode Parsing in JavaScript/TypeScript

5.1 Library Comparison

Library	Browser Support	Buffer-free	TypeScript	Size	Notes
`@ctrl/torrent-file`	Native (Uint8Array)	Yes	Yes	Small	Purpose-built, no Buffer dependency
`@substrate-system/bencode`	Native (Uint8Array)	Yes	Yes	Small	Node.js Buffer replaced with Uint8Array
`bencode-js` (benjreinhart)	Native	Yes	No	Tiny	Zero dependency, works in all environments
`parse-torrent`	Browserify/ webpack needed	No (uses Buffer)	Yes	Medium	Most popular, widely used by WebTorrent
`node-bencode` (webtorrent)	Browserify needed	No	Yes	Medium	Requires Buffer polyfill in browser

Recommendation for browser extensions: Use `@ctrl/torrent-file` or a custom implementation based on `bencode-js` that works directly with `Uint8Array`/`ArrayBuffer` without Node.js Buffer dependencies.

5.2 Using @ctrl/torrent-file (Recommended)

Claim: `@ctrl/torrent-file` implements its own bencode encoder/decoder that does not use `Buffer`, making it easier to use in browser environments. **Source:** @ctrl/torrent-file GitHub README **URL:** https://github.com/scttcper/torrent-file **Excerpt:** > “This project is based on parse-torrent and node-bencode to parse the data of a torrent file. This library implements its own bencode encoder and decoder that does not use `Buffer` making it easier to use in browser or non node environments.” **Confidence:** high

```
// Using @ctrl/torrent-file in browser extension
import { info, files, hash, hashes, toTorrentFile } from '@ctrl/
        torrent-file';

// Parse torrent info from Uint8Array
function parseTorrentFile(data: Uint8Array) {
  const torrentInfo = info(data);
  const torrentFiles = files(data);
  const infoHash = hash(data);
  const allHashes = hashes(data); // { version, infoHash,
        infoHashV2? }

  return {
    infoHash,
    name: torrentInfo.name,
    announce: torrentInfo.announce,
    files: torrentFiles.files,
    pieceLength: torrentInfo.pieceLength,
    totalSize: torrentFiles.length,
    isPrivate: torrentInfo.private === 1
  };
}
```

5.3 Custom Bencode Decoder (Zero Dependencies)

For minimal dependency footprint, implement a custom bencode decoder directly on `Uint8Array`:

```
/**
 * Bencode decoder using Uint8Array (zero dependencies, browser-
        native)
 *
 * Handles: byte strings, integers, lists, dictionaries
 * Returns: native JS types with Uint8Array for binary strings
 */

export type BencodeValue = string | number | BencodeValue[] |
        Record<string, BencodeValue> | Uint8Array;

export class BencodeDecoder {
  private data: Uint8Array;
  private pos: number = 0;

  constructor(data: Uint8Array) {
    this.data = data;
  }

  /**
   * Decode the entire buffer
   */
  decode(): BencodeValue {
```

```
  const result = this.decodeNext();
  if (this.pos !== this.data.length) {
    throw new Error(`Extra data at position ${this.pos}`);
  }
  return result;
}

/**
 * Decode the next value at current position
 */
private decodeNext(): BencodeValue {
  if (this.pos >= this.data.length) {
    throw new Error('Unexpected end of data');
  }

  const ch = this.data[this.pos];

  if (ch === 0x69) { // 'i'
    return this.decodeInteger();
  } else if (ch === 0x6C) { // 'l'
    return this.decodeList();
  } else if (ch === 0x64) { // 'd'
    return this.decodeDictionary();
  } else if (ch === 0x65) { // 'e'
    throw new Error(`Unexpected 'e' at position ${this.pos}`);
  } else if (ch >= 0x30 && ch <= 0x39) { // '0'-'9'
    return this.decodeByteString();
  } else {
    throw new Error(`Invalid bencode: unexpected byte 0x$
      {ch.toString(16)} at position ${this.pos}`);
  }
}

/**
 * Decode an integer: i<number>e
 */
private decodeInteger(): number {
  if (this.data[this.pos] !== 0x69) {
    throw new Error('Expected "i" for integer');
  }
  this.pos++; // skip 'i'

  const endPos = this.findByte(0x65); // 'e'
  const numStr = this.bytesToString(this.data, this.pos,
      endPos);

  // Validate: no leading zeros (except for 0 itself)
  if (numStr.length > 1 && numStr[0] === '0') {
    throw new Error('Leading zeros not allowed in integer');
  }
  if (numStr === '-0') {
    throw new Error('Negative zero is not valid');
```

```
  }

  const value = parseInt(numStr, 10);
  if (isNaN(value)) {
    throw new Error(`Invalid integer: ${numStr}`);
  }

  this.pos = endPos + 1; // skip past 'e'
  return value;
}

/**
 * Decode a byte string: <length>:<content>
 * Returns Uint8Array for raw binary data
 */
private decodeByteString(): Uint8Array {
  const colonPos = this.findByte(0x3A); // ':'
  const lenStr = this.bytesToString(this.data, this.pos,
      colonPos);
  const length = parseInt(lenStr, 10);

  if (isNaN(length) || length < 0) {
    throw new Error(`Invalid string length: ${lenStr}`);
  }

  this.pos = colonPos + 1; // skip past ':'

  if (this.pos + length > this.data.length) {
    throw new Error(`String extends beyond data: need ${length}
      bytes at position ${this.pos}`);
  }

  const value = this.data.slice(this.pos, this.pos + length);
  this.pos += length;
  return value;
}

/**
 * Decode a list: l<values>e
 */
private decodeList(): BencodeValue[] {
  if (this.data[this.pos] !== 0x6C) {
    throw new Error('Expected "l" for list');
  }
  this.pos++; // skip 'l'

  const list: BencodeValue[] = [];
  while (this.pos < this.data.length && this.data[this.pos] !==
      0x65) {
    list.push(this.decodeNext());
  }
```

```
  if (this.pos >= this.data.length) {
    throw new Error('Unterminated list');
  }
  this.pos++; // skip 'e'
  return list;
}

/**
 * Decode a dictionary: d<key><value>...e
 */
private decodeDictionary(): Record<string, BencodeValue> {
  if (this.data[this.pos] !== 0x64) {
    throw new Error('Expected "d" for dictionary');
  }
  this.pos++; // skip 'd'

  const dict: Record<string, BencodeValue> = {};
  let lastKey: string | null = null;

  while (this.pos < this.data.length && this.data[this.pos] !==
      0x65) {
    // Keys are always byte strings
    const keyBytes = this.decodeByteString();
    const key = new TextDecoder().decode(keyBytes);

    // Validate key ordering (critical for infohash computation)
    if (lastKey !== null && key < lastKey) {
      throw new Error(`Dictionary keys out of order: "${key}" <
      "${lastKey}"`);
    }
    lastKey = key;

    const value = this.decodeNext();
    dict[key] = value;
  }

  if (this.pos >= this.data.length) {
    throw new Error('Unterminated dictionary');
  }
  this.pos++; // skip 'e'
  return dict;
}

/**
 * Find the position of a specific byte, starting from current
      position
 */
private findByte(target: number): number {
  for (let i = this.pos; i < this.data.length; i++) {
    if (this.data[i] === target) return i;
  }
```

```
    throw new Error(`Could not find byte 0x${target.toString(16)}
        `);
  }

  /**
   * Convert a range of bytes to a string (ASCII)
   */
  private bytesToString(data: Uint8Array, start: number, end:
        number): string {
    let result = '';
    for (let i = start; i < end; i++) {
      result += String.fromCharCode(data[i]);
    }
    return result;
  }
}

/**
 * Convenience function to decode bencode data
 */
export function decodeBencode(data: Uint8Array): BencodeValue {
  return new BencodeDecoder(data).decode();
}
```

5.4 Custom Bencode Encoder

```
/**
 * Bencode encoder using Uint8Array (zero dependencies)
 */

export class BencodeEncoder {
  private chunks: Uint8Array[] = [];

  /**
   * Encode a JavaScript value to bencode
   */
  encode(value: BencodeValue): Uint8Array {
    this.chunks = [];
    this.encodeValue(value);

    // Concatenate all chunks
    const totalLength = this.chunks.reduce((sum, c) => sum +
        c.length, 0);
    const result = new Uint8Array(totalLength);
    let offset = 0;
    for (const chunk of this.chunks) {
      result.set(chunk, offset);
      offset += chunk.length;
    }
    return result;
  }
```

```
private encodeValue(value: BencodeValue): void {
  if (value === null || value === undefined) {
    throw new Error('Cannot encode null/undefined');
  }

  if (typeof value === 'number') {
    this.encodeInteger(value);
  } else if (typeof value === 'string') {
    this.encodeString(value);
  } else if (value instanceof Uint8Array) {
    this.encodeByteString(value);
  } else if (Array.isArray(value)) {
    this.encodeList(value);
  } else if (typeof value === 'object') {
    this.encodeDictionary(value as Record<string,
      BencodeValue>);
  } else {
    throw new Error(`Cannot encode type: ${typeof value}`);
  }
}

private encodeInteger(value: number): void {
  if (!Number.isInteger(value)) {
    throw new Error('Only integers can be encoded');
  }
  this.chunks.push(stringToUint8Array(`i${value}e`));
}

private encodeString(value: string): void {
  const bytes = new TextEncoder().encode(value);
  this.encodeByteString(bytes);
}

private encodeByteString(value: Uint8Array): void {
  const prefix = stringToUint8Array(`${value.length}:`);
  this.chunks.push(prefix, value);
}

private encodeList(value: BencodeValue[]): void {
  this.chunks.push(new Uint8Array([0x6C])); // 'l'
  for (const item of value) {
    this.encodeValue(item);
  }
  this.chunks.push(new Uint8Array([0x65])); // 'e'
}

private encodeDictionary(value: Record<string, BencodeValue>):
      void {
  this.chunks.push(new Uint8Array([0x64])); // 'd'

  // Keys must be sorted lexicographically
```

```
    const keys = Object.keys(value).sort();
    for (const key of keys) {
      if (value[key] === undefined || value[key] === null)
        continue;
      this.encodeString(key);
      this.encodeValue(value[key]);
    }

    this.chunks.push(new Uint8Array([0x65])); // 'e'
  }
}

function stringToUint8Array(str: string): Uint8Array {
  return new TextEncoder().encode(str);
}

/**
 * Convenience function to encode a value to bencode
 */
export function encodeBencode(value: BencodeValue): Uint8Array {
  return new BencodeEncoder().encode(value);
}
```

---

6. Computing Infohash from Parsed Torrent

6.1 Infohash Definition

Claim: The infohash is the SHA-1 hash of the **bencoded info dictionary** as it appears in the .torrent file (not a re-encoded version). **Source:** BEP 3 / Stack Overflow authoritative answer **URL:** https://stackoverflow.com/questions/46025771/python3-calculating-torrent-hash **Excerpt:** > “The hash in a torrent client or the hash you find in a magnet-URI is the SHA1-hash of the raw bencoded info-dictionary-part of a torrent-file.” **Confidence:** high

Critical implementation note: > “Conversely that means implementations must either reject invalid metainfo files or extract the substring directly. They must not perform a decode-encode roundtrip on invalid data.” — BEP 52 [281]

This means the most reliable method is to **extract the raw bencoded bytes of the info dictionary** from the original torrent file, rather than decoding and re-encoding (which could produce different byte ordering).

6.2 Infohash Computation with Extraction

```
/**
 * Extract the raw bencoded info dictionary bytes and compute
       SHA-1 infohash
 *
 * Strategy: Parse just enough to find the info dictionary
       boundaries,
 * then extract the raw bytes and hash them.
 */

/**
 * Extract the bencoded info dictionary as raw bytes from a
       torrent file
 * This is the MOST RELIABLE method for computing the infohash
 */
function extractInfoDict(data: Uint8Array): { infoDict:
       Uint8Array; start: number; end: number } {
  // The top-level torrent is a dictionary that must contain an
       'info' key
  // Format: d...4:info<info_dict>e...e

  // We need to find the 'info' key and then the matching
       dictionary
  const infoKeyStr = '4:info';
  const infoKeyBytes = new TextEncoder().encode(infoKeyStr);

  // Find the '4:info' key in the top-level dictionary
  let infoKeyPos = -1;
  for (let i = 0; i <= data.length - infoKeyBytes.length; i++) {
    let match = true;
    for (let j = 0; j < infoKeyBytes.length; j++) {
      if (data[i + j] !== infoKeyBytes[j]) {
        match = false;
        break;
      }
    }
    if (match) {
      infoKeyPos = i;
      break;
    }
  }

  if (infoKeyPos === -1) {
    throw new Error('Could not find "info" key in torrent file');
  }

  // The info dictionary starts right after "4:info"
  const dictStart = infoKeyPos + infoKeyBytes.length;

  // Verify it starts with 'd'
  if (data[dictStart] !== 0x64) {
```

```
    throw new Error('Expected dictionary after "info" key');
  }

  // Find the matching 'e' for this dictionary by tracking nesting
  let depth = 0;
  let dictEnd = dictStart;

  while (dictEnd < data.length) {
    const ch = data[dictEnd];

    if (ch >= 0x30 && ch <= 0x39) {
      // Byte string: read length, skip content
      let colonPos = dictEnd;
      while (colonPos < data.length && data[colonPos] !== 0x3A)
        colonPos++;
      const lenStr = String.fromCharCode(...data.slice(dictEnd,
        colonPos));
      const len = parseInt(lenStr, 10);
      dictEnd = colonPos + 1 + len; // skip past string
      continue;
    } else if (ch === 0x64 || ch === 0x6C) { // 'd' or 'l'
      depth++;
    } else if (ch === 0x65) { // 'e'
      depth--;
      if (depth === 0) {
        dictEnd++; // include the 'e'
        break;
      }
    } else if (ch === 0x69) { // 'i'
      // Integer: skip to 'e'
      while (dictEnd < data.length && data[dictEnd] !== 0x65)
        dictEnd++;
      dictEnd++; // include 'e'
      continue;
    }

    dictEnd++;
  }

  if (depth !== 0) {
    throw new Error('Unterminated info dictionary');
  }

  return {
    infoDict: data.slice(dictStart, dictEnd),
    start: dictStart,
    end: dictEnd
  };
}

/**
 * Compute SHA-1 infohash using Web Crypto API
```

```
 */
async function computeInfoHash(infoDictBytes: Uint8Array):
        Promise<string> {
  const hashBuffer = await crypto.subtle.digest('SHA-1',
        infoDictBytes);
  const hashArray = new Uint8Array(hashBuffer);

  // Convert to hex string
  return Array.from(hashArray)
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');
}

/**
 * Complete: Compute infohash from torrent file data
 */
export async function getInfoHashFromTorrent(data: Uint8Array):
        Promise<string> {
  const { infoDict } = extractInfoDict(data);
  return computeInfoHash(infoDict);
}
```

6.3 Alternative: Using @ctrl/torrent-file

```
import { hash } from '@ctrl/torrent-file';

// Simple one-liner
const infoHash = hash(torrentUint8Array);
console.log(infoHash); // e.g.,
        "d2474e86c95b19b8bcfdb92bc12c9d44667cfa36"
```

6.4 Alternative: Using parse-torrent (with Buffer polyfill)

```
import parseTorrent from 'parse-torrent';

// Requires Buffer polyfill in browser (webpack/rollup config)
const parsed = parseTorrent(Buffer.from(torrentUint8Array));
console.log(parsed.infoHash); // hex string
```

Note: `parse-torrent` requires Node.js `Buffer`. For browser use, you need to polyfill with `feross/buffer`:

```
// webpack/vite config
import { Buffer } from 'buffer/';
window.Buffer = Buffer;
```

---

7. Torrent File Structure: Single vs Multi-File

7.1 Single-File Torrent

```
{
  'announce': 'http://bttracker.debian.org:6969/announce',
  'info': {
    'length': 678301696,           // File size in bytes
    'name': 'debian-503-amd64-CD-1.iso',  // Suggested filename
    'piece length': 262144,        // 256 KB per piece
    'pieces': <binary SHA1 hashes> // 20 bytes * num_pieces
  }
}
```

Source: Wikipedia - Torrent file **URL:** https://en.wikipedia.org/wiki/Torrent_file **Confidence:** high

7.2 Multi-File Torrent

```
{
  'announce': 'http://tracker.example.com/announce',
  'info': {
    'name': 'directoryName',       // Directory name
    'piece length': 262144,        // 256 KB per piece
    'pieces': <binary SHA1 hashes>,
    'files': [                     // List of files
      { 'length': 111, 'path': ['111.txt'] },
      { 'length': 222, 'path': ['subdir', '222.txt'] }
    ]
  }
}
```

7.3 Computing Total Size

```
function getTotalSize(info: Record<string, BencodeValue>): number
        {
  if ('length' in info) {
    // Single-file torrent
    return info['length'] as number;
  } else if ('files' in info) {
    // Multi-file torrent
    const files = info['files'] as Array<Record<string,
        BencodeValue>>;
    return files.reduce((total, file) => total + (file['length']
        as number), 0);
  }
  throw new Error('Cannot determine torrent size');
}
```

7.4 File Path Construction

```
interface TorrentFile {
  path: string;      // Full path (e.g., "directoryName/subdir/
        file.txt")
  name: string;      // Filename only
  length: number;    // File size in bytes
  offset: number;    // Byte offset in the concatenated stream
}

function getFiles(info: Record<string, BencodeValue>):
        TorrentFile[] {
  const name = uint8ArrayToString(info['name'] as Uint8Array);

  if ('length' in info) {
    // Single file
    return [{
      path: name,
      name,
      length: info['length'] as number,
      offset: 0
    }];
  }

  // Multi-file
  const files = info['files'] as Array<Record<string,
        BencodeValue>>;
  let offset = 0;

  return files.map(file => {
    const pathParts = (file['path'] as Uint8Array[]).map(p =>
        uint8ArrayToString(p));
    const fileName = pathParts[pathParts.length - 1];
    const fullPath = [name, ...pathParts].join('/');
    const length = file['length'] as number;

    const result: TorrentFile = {
      path: fullPath,
      name: fileName,
      length,
      offset
    };
    offset += length;
    return result;
  });
}

function uint8ArrayToString(data: Uint8Array | string): string {
  if (typeof data === 'string') return data;
  return new TextDecoder().decode(data);
}
```

---

8. Piece Hash Validation

8.1 Piece Structure

The `pieces` field in the info dictionary contains the concatenation of all 20-byte SHA-1 hashes, one per piece.

```
/**
 * Extract individual piece hashes from the pieces blob
 */
function extractPieceHashes(pieces: Uint8Array): Uint8Array[] {
  const PIECE_HASH_SIZE = 20; // SHA-1 = 20 bytes

  if (pieces.length % PIECE_HASH_SIZE !== 0) {
    throw new Error(`Invalid pieces length: ${pieces.length} is
        not a multiple of ${PIECE_HASH_SIZE}`);
  }

  const numPieces = pieces.length / PIECE_HASH_SIZE;
  const hashes: Uint8Array[] = [];

  for (let i = 0; i < numPieces; i++) {
    const offset = i * PIECE_HASH_SIZE;
    hashes.push(pieces.slice(offset, offset + PIECE_HASH_SIZE));
  }

  return hashes;
}

/**
 * Verify a piece of downloaded data against its expected hash
 */
async function verifyPiece(pieceData: Uint8Array, expectedHash:
        Uint8Array): Promise<boolean> {
  const hashBuffer = await crypto.subtle.digest('SHA-1',
        pieceData);
  const actualHash = new Uint8Array(hashBuffer);

  // Constant-time comparison
  if (actualHash.length !== expectedHash.length) return false;
  let match = 0;
  for (let i = 0; i < actualHash.length; i++) {
    match |= actualHash[i] ^ expectedHash[i];
  }
  return match === 0;
}

/**
 * Get the expected size of the last piece
 */
```

```
function getLastPieceLength(totalSize: number, pieceLength:
        number): number {
  return totalSize - (Math.floor((totalSize - 1) / pieceLength) *
        pieceLength);
}
```

---

9. Creating Magnet URI from Torrent File

9.1 Magnet URI Format

The magnet URI format is defined in BEP 9 and follows this structure:

```
magnet:?
xt=urn:btih:<INFOHASH>&dn=<DISPLAY_NAME>&tr=<TRACKER_URL>&tr=<TRACK
```

Source: Wikipedia - Magnet URI scheme **URL:** https://en.wikipedia.org/wiki/Magnet_URI_scheme **Confidence:** high

9.2 Parameters Reference

Parameter	Name	Required	Description
`xt`	eXact Topic	Yes	URN with infohash. Format: `urn:btih:<40-char-hex>`
`dn`	Display Name	No	Human-readable filename
`tr`	TRacker	No	Tracker URL (can have multiple). Must be URL-encoded
`xl`	eXact Length	No	Total file size in bytes
`ws`	Web Seed	No	HTTP(S) URL for web seeding (BEP 19)
`xs`	eXact Source	No	Direct link to .torrent file

9.3 Magnet URI Generation Code

```
/**
 * Generate a magnet URI from parsed torrent metadata
```

```
 */
function generateMagnetUri(options: {
  infoHash: string;
  name?: string;
  trackers?: string[];
  size?: number;
  webSeeds?: string[];
}): string {
  const params = new URLSearchParams();

  // xt = exact topic (required)
  params.set('xt', `urn:btih:${options.infoHash.toLowerCase()}`);

  // dn = display name
  if (options.name) {
    params.set('dn', options.name);
  }

  // xl = exact length
  if (options.size !== undefined) {
    params.set('xl', String(options.size));
  }

  // tr = trackers (can be multiple)
  if (options.trackers) {
    for (const tracker of options.trackers) {
      params.append('tr', tracker);
    }
  }

  // ws = web seeds (can be multiple)
  if (options.webSeeds) {
    for (const seed of options.webSeeds) {
      params.append('ws', seed);
    }
  }

  return `magnet:?${params.toString()}`;
}

/**
 * Generate magnet URI from raw torrent data
 */
export async function torrentToMagnet(data: Uint8Array):
        Promise<string> {
  // Decode the torrent
  const decoded = decodeBencode(data) as Record<string,
        BencodeValue>;
  const info = decoded['info'] as Record<string, BencodeValue>;

  // Extract infohash
  const { infoDict } = extractInfoDict(data);
```

```
  const infoHash = await computeInfoHash(infoDict);

  // Extract name
  const name = new TextDecoder().decode(info['name'] as
        Uint8Array);

  // Extract trackers
  const trackers: string[] = [];
  if ('announce' in decoded) {
    trackers.push(new TextDecoder().decode(decoded['announce'] as
        Uint8Array));
  }
  if ('announce-list' in decoded) {
    const announceList = decoded['announce-list'] as
        BencodeValue[][];
    for (const tier of announceList) {
      for (const tracker of tier) {
        const trackerStr = tracker instanceof Uint8Array
          ? new TextDecoder().decode(tracker)
          : String(tracker);
        if (!trackers.includes(trackerStr)) {
          trackers.push(trackerStr);
        }
      }
    }
  }

  // Extract size
  const totalSize = getTotalSize(info);

  return generateMagnetUri({
    infoHash,
    name,
    trackers,
    size: totalSize
  });
}
```

9.4 Using magnet-uri Library

```
import magnet from 'magnet-uri';

// Encode a parsed torrent to magnet URI
const uri = magnet.encode({
  xt: 'urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36',
  dn: 'My File.txt',
  xl: '10826029',
  tr: ['udp://tracker.openbittorrent.com:80/announce']
});

// Decode a magnet URI
```

```
const parsed = magnet.decode('magnet:?xt=urn:btih:...');
console.log(parsed.infoHash); // The infohash
```

Source: webtorrent/magnet-uri GitHub **URL:** https://github.com/webtorrent/magnet-uri **Confidence:** high

---

10. Private Torrent Handling

10.1 Private Flag (BEP 27)

Claim: When a torrent contains `info.private = 1`, the client MUST disable DHT, PEX, and LSD for that torrent. **Source:** BEP 27 - Private Torrents **URL:** https://www.bittorrent.org/beps/bep_0027.html **Excerpt:** > “When a BitTorrent client obtains a metainfo file containing the ‘private=1’ key-value pair, it MUST ONLY announce itself to the private tracker, and MUST ONLY initiate connections to peers returned from the private tracker.” **Confidence:** high

10.2 Private Flag Detection

```
/**
 * Check if a torrent is private
 */
function isPrivateTorrent(info: Record<string, BencodeValue>):
        boolean {
  return info['private'] === 1;
}

/**
 * Get implications of private flag
 */
function getPrivateTorrentBehavior(info: Record<string,
        BencodeValue>): {
  disableDHT: boolean;
  disablePEX: boolean;
  disableLSD: boolean;
  trackerOnly: boolean;
} {
  const isPrivate = isPrivateTorrent(info);
  return {
    disableDHT: isPrivate,
    disablePEX: isPrivate,
    disableLSD: isPrivate,
    trackerOnly: isPrivate
  };
}
```

10.3 Passkey in Announce URL

Private trackers embed a unique passkey in the announce URL to identify the user:

```
https://tracker.example.com/UNIQUE_PASSKEY/announce
```

This passkey should be treated as sensitive data — never share or log it:

```
/**
 * Extract tracker domain from announce URL (without passkey)
 */
function getTrackerDomain(announceUrl: string): string {
  try {
    const url = new URL(announceUrl);
    return url.hostname;
  } catch {
    return announceUrl;
  }
}

/**
 * Sanitize announce URL for logging (remove passkey)
 */
function sanitizeAnnounceUrl(announceUrl: string): string {
  try {
    const url = new URL(announceUrl);
    // Remove potential passkey from path
    const pathParts = url.pathname.split('/').filter(p =>
        p.length > 0);
    if (pathParts.length >= 2 && pathParts[0].length === 32) {
      // Likely passkey pattern
      pathParts[0] = '***PASSKEY***';
      url.pathname = '/' + pathParts.join('/');
    }
    return url.toString();
  } catch {
    return '[invalid-url]';
  }
}
```

---

11. Torrent File Validation

11.1 Required Fields Validation

```
interface ValidationResult {
  valid: boolean;
```

```
  errors: string[];
  warnings: string[];
}

/**
 * Validate a parsed torrent structure
 */
function validateTorrent(decoded: Record<string, BencodeValue>):
        ValidationResult {
  const errors: string[] = [];
  const warnings: string[] = [];

  // Check top-level structure
  if (!('info' in decoded)) {
    errors.push('Missing required "info" dictionary');
  }
  if (!('announce' in decoded) && !('announce-list' in decoded)) {
    errors.push('Missing tracker: neither "announce" nor
        "announce-list" present');
  }

  // Validate info dictionary
  if ('info' in decoded) {
    const info = decoded['info'] as Record<string, BencodeValue>;

    // Required info fields
    if (!('name' in info)) {
      errors.push('Missing required "info.name"');
    }
    if (!('piece length' in info)) {
      errors.push('Missing required "info.piece length"');
    }
    if (!('pieces' in info)) {
      errors.push('Missing required "info.pieces"');
    } else {
      const pieces = info['pieces'] as Uint8Array;
      if (pieces.length % 20 !== 0) {
        errors.push(`Invalid pieces length: $
        {pieces.length} (must be multiple of 20)`);
      }
    }

    // Must have either 'length' (single-file) or 'files' (multi-
        file)
    if (!('length' in info) && !('files' in info)) {
      errors.push('Missing either "info.length" (single-file) or
        "info.files" (multi-file)');
    }
    if ('length' in info && 'files' in info) {
      warnings.push('Both "length" and "files" present (should
        only have one)');
    }
```

```
    // Validate piece length is reasonable
    if ('piece length' in info) {
      const pieceLength = info['piece length'] as number;
      const validPieceLengths = [32768, 65536, 131072, 262144,
        524288, 1048576, 2097152, 4194304, 8388608];
      if (!validPieceLengths.includes(pieceLength)) {
        warnings.push(`Non-standard piece length: ${pieceLength}
        bytes`);
      }
    }

    // Validate files structure (multi-file)
    if ('files' in info) {
      const files = info['files'] as Array<Record<string,
        BencodeValue>>;
      if (!Array.isArray(files)) {
        errors.push('"info.files" must be a list');
      } else {
        for (let i = 0; i < files.length; i++) {
          const file = files[i];
          if (!('length' in file)) {
            errors.push(`File ${i}: missing "length"`);
          }
          if (!('path' in file)) {
            errors.push(`File ${i}: missing "path"`);
          }
        }
      }
    }
  }

  // Validate announce URL
  if ('announce' in decoded) {
    const announce = decoded['announce'];
    const announceStr = announce instanceof Uint8Array
      ? new TextDecoder().decode(announce)
      : String(announce);
    try {
      new URL(announceStr);
    } catch {
      errors.push('Invalid announce URL');
    }
  }

  return {
    valid: errors.length === 0,
    errors,
    warnings
  };
}
```

---

12. Blob/ArrayBuffer Handling in Browser Extensions

12.1 Reading Binary Data from fetch()

```
/**
 * Fetch a .torrent file and convert to Uint8Array
 */
async function fetchTorrentFile(url: string): Promise<Uint8Array>
        {
  const response = await fetch(url, {
    headers: { Accept: 'application/x-bittorrent,*/*' }
  });

  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }

  // Get as ArrayBuffer, then convert to Uint8Array
  const arrayBuffer = await response.arrayBuffer();
  return new Uint8Array(arrayBuffer);
}
```

12.2 Converting Between Binary Types

```
/**
 * Convert various input types to Uint8Array
 */
function toUint8Array(data: ArrayBuffer | Uint8Array | Blob |
        number[]): Uint8Array {
  if (data instanceof Uint8Array) {
    return data;
  }
  if (data instanceof ArrayBuffer) {
    return new Uint8Array(data);
  }
  if (data instanceof Blob) {
    // Need async: blob.arrayBuffer() -> Uint8Array
    throw new Error('Blob conversion requires async: use
        blobToUint8Array()');
  }
  if (Array.isArray(data)) {
    return new Uint8Array(data);
  }
  throw new Error(`Cannot convert type to Uint8Array: ${typeof
        data}`);
}

/**
```

```
 * Async conversion from Blob to Uint8Array
 */
async function blobToUint8Array(blob: Blob): Promise<Uint8Array> {
  const arrayBuffer = await blob.arrayBuffer();
  return new Uint8Array(arrayBuffer);
}

/**
 * Convert Uint8Array to hex string
 */
function uint8ArrayToHex(data: Uint8Array): string {
  return Array.from(data)
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');
}

/**
 * Convert hex string to Uint8Array
 */
function hexToUint8Array(hex: string): Uint8Array {
  if (hex.length % 2 !== 0) {
    throw new Error('Hex string must have even length');
  }
  const bytes = new Uint8Array(hex.length / 2);
  for (let i = 0; i < hex.length; i += 2) {
    bytes[i / 2] = parseInt(hex.substring(i, i + 2), 16);
  }
  return bytes;
}

/**
 * Compare two Uint8Arrays (constant-time)
 */
function constantTimeEquals(a: Uint8Array, b: Uint8Array):
        boolean {
  if (a.length !== b.length) return false;
  let result = 0;
  for (let i = 0; i < a.length; i++) {
    result |= a[i] ^ b[i];
  }
  return result === 0;
}
```

12.3 Handling Large Torrent Files

```
/**
 * Parse a torrent file with size validation
 */
function parseTorrentSafe(data: Uint8Array, maxSize: number = 50
        * 1024 * 1024): {
  decoded: Record<string, BencodeValue>;
```

```
  infoHash: Promise<string>;
} {
  // Validate size (most .torrent files are under 1MB)
  if (data.length > maxSize) {
    throw new Error(`Torrent file too large: ${data.length} bytes
        (max: ${maxSize})`);
  }

  // Validate it starts with 'd' (dictionary)
  if (data.length === 0 || data[0] !== 0x64) {
    throw new Error('Invalid torrent file: must start with a
        dictionary');
  }

  const decoded = decodeBencode(data) as Record<string,
        BencodeValue>;

  // Validate structure
  const validation = validateTorrent(decoded);
  if (!validation.valid) {
    throw new Error(`Invalid torrent: ${validation.errors.join(',
        ')}`);
  }

  // Log warnings
  if (validation.warnings.length > 0) {
    console.warn('Torrent warnings:', validation.warnings);
  }

  const infoHashPromise = (async () => {
    const { infoDict } = extractInfoDict(data);
    return computeInfoHash(infoDict);
  })();

  return { decoded, infoHash: infoHashPromise };
}
```

---

13. Complete Implementation

13.1 Complete Torrent Parser Class

```
/**
 * Complete torrent file parser for browser extensions
 * Zero dependencies - uses only Web APIs
 */

export interface ParsedTorrent {
  infoHash: string;
  name: string;
```

```
  announce: string[];
  pieceLength: number;
  totalSize: number;
  files: Array<{
    path: string;
    name: string;
    length: number;
    offset: number;
  }>;
  pieces: Uint8Array[];       // Individual piece hashes
  isPrivate: boolean;
  comment?: string;
  createdBy?: string;
  creationDate?: Date;
  magnetUri: string;
}

export class TorrentParser {
  /**
   * Parse a torrent file from Uint8Array data
   */
  static async parse(data: Uint8Array): Promise<ParsedTorrent> {
    // Validate
    if (data.length === 0) throw new Error('Empty torrent data');
    if (data[0] !== 0x64) throw new Error('Not a valid torrent
        file (must start with dict)');

    // Decode bencode
    const decoded = decodeBencode(data) as Record<string,
        BencodeValue>;
    const info = decoded['info'] as Record<string, BencodeValue>;

    // Validate structure
    const validation = validateTorrent(decoded);
    if (!validation.valid) {
      throw new Error(`Invalid torrent: $
        {validation.errors.join(', ')}`);
    }

    // Extract infohash (most critical)
    const { infoDict } = extractInfoDict(data);
    const infoHash = await computeInfoHash(infoDict);

    // Extract name
    const name = bytesToString(info['name'] as Uint8Array);

    // Extract trackers
    const announce: string[] = [];
    if (decoded['announce']) {
      announce.push(bytesToString(decoded['announce'] as
        Uint8Array));
    }
```

```
if (decoded['announce-list']) {
  const tiers = decoded['announce-list'] as BencodeValue[][];
  for (const tier of tiers) {
    for (const t of tier) {
      const url = bytesToString(t as Uint8Array);
      if (!announce.includes(url)) announce.push(url);
    }
  }
}

// Extract piece info
const pieceLength = info['piece length'] as number;
const piecesBlob = info['pieces'] as Uint8Array;
const pieces = extractPieceHashes(piecesBlob);

// Extract file info
const totalSize = getTotalSize(info);
const files = getFiles(info);

// Check private flag
const isPrivate = info['private'] === 1;

// Extract optional fields
const comment = decoded['comment']
  ? bytesToString(decoded['comment'] as Uint8Array)
  : undefined;
const createdBy = decoded['created by']
  ? bytesToString(decoded['created by'] as Uint8Array)
  : undefined;
const creationDate = decoded['creation date']
  ? new Date((decoded['creation date'] as number) * 1000)
  : undefined;

// Generate magnet URI
const magnetUri = generateMagnetUri({
  infoHash,
  name,
  trackers: announce,
  size: totalSize
});

return {
  infoHash,
  name,
  announce,
  pieceLength,
  totalSize,
  files,
  pieces,
  isPrivate,
  comment,
  createdBy,
```

```
      creationDate,
      magnetUri
    };
  }

  /**
   * Compute infohash only (fast path when full parsing isn't
       needed)
   */
  static async getInfoHash(data: Uint8Array): Promise<string> {
    const { infoDict } = extractInfoDict(data);
    return computeInfoHash(infoDict);
  }

  /**
   * Check if data appears to be a valid torrent file (without
       full parsing)
   */
  static isTorrentFile(data: Uint8Array): boolean {
    if (data.length < 10) return false;
    if (data[0] !== 0x64) return false; // must start with 'd'

    // Quick check: does it contain "4:info"?
    const infoKey = new TextEncoder().encode('4:info');
    outer: for (let i = 0; i <= data.length - infoKey.length; i+
        +) {
      for (let j = 0; j < infoKey.length; j++) {
        if (data[i + j] !== infoKey[j]) continue outer;
      }
      return true; // Found "4:info"
    }
    return false;
  }
}

// Helper
function bytesToString(data: Uint8Array | string): string {
  return typeof data === 'string' ? data : new
      TextDecoder().decode(data);
}
```

13.2 Service Worker Background Script

```
// background.ts

import { TorrentParser, type ParsedTorrent } from './torrent-
    parser';

// Message types
interface TorrentDownloadMessage {
  action: 'downloadTorrent';
  url: string;
```

```
  timeout?: number;
}

interface ParseTorrentMessage {
  action: 'parseTorrent';
  data: number[]; // JSON-serializable: Uint8Array as array
}

interface TorrentInfoHashMessage {
  action: 'getInfoHash';
  data: number[];
}

type ExtensionMessage = TorrentDownloadMessage |
        ParseTorrentMessage | TorrentInfoHashMessage;

// Message handler
chrome.runtime.onMessage.addListener(
  (
    request: ExtensionMessage,
    sender: chrome.runtime.MessageSender,
    sendResponse: (response: any) => void
  ): boolean => {
    handleMessage(request, sender)
      .then(sendResponse)
      .catch(error => {
        sendResponse({
          success: false,
          error: error instanceof Error ? error.message :
        String(error)
        });
      });
    return true; // Async response
  }
);

async function handleMessage(
  request: ExtensionMessage,
  sender: chrome.runtime.MessageSender
): Promise<any> {
  switch (request.action) {
    case 'downloadTorrent': {
      const response = await fetch(request.url, {
        headers: { Accept: 'application/x-bittorrent,*/*' },
        signal: AbortSignal.timeout(request.timeout || 30000)
      });

      if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
      }

      const arrayBuffer = await response.arrayBuffer();
```

```
      const data = new Uint8Array(arrayBuffer);

      // Parse the torrent
      const parsed = await TorrentParser.parse(data);

      return {
        success: true,
        infoHash: parsed.infoHash,
        magnetUri: parsed.magnetUri,
        name: parsed.name,
        totalSize: parsed.totalSize,
        isPrivate: parsed.isPrivate,
        announce: parsed.announce
      };
    }

    case 'parseTorrent': {
      const data = new Uint8Array(request.data);
      const parsed = await TorrentParser.parse(data);
      return {
        success: true,
        ...parsed,
        // Uint8Arrays need to be converted for JSON serialization
        pieces: parsed.pieces.map(p => Array.from(p))
      };
    }

    case 'getInfoHash': {
      const data = new Uint8Array(request.data);
      const infoHash = await TorrentParser.getInfoHash(data);
      return { success: true, infoHash };
    }

    default:
      throw new Error(`Unknown action: ${(request as any).action}
        `);
  }
}

// Optional: Handle magnet: protocol clicks
chrome.runtime.onInstalled.addListener(() => {
  // Register protocol handler if needed
  console.log('Torrent parser extension installed');
});
```

13.3 Content Script Integration

```
// content.ts

/**
 * Scan page for torrent links and magnet URIs
```

```typescript
 */
function scanPage(): Array<{ url: string; type: 'torrent' |
        'magnet'; text: string }> {
  const results: Array<{ url: string; type: 'torrent' | 'magnet';
        text: string }> = [];
  const links = document.querySelectorAll('a[href]');

  for (const link of links) {
    const el = link as HTMLAnchorElement;
    const href = el.href || '';
    const text = el.textContent?.trim() || '';

    if (href.startsWith('magnet:?xt=urn:btih:')) {
      results.push({ url: href, type: 'magnet', text });
    } else if (/\.torrent(?:\?.*)?(?:#.*)?$/i.test(href)) {
      results.push({ url: href, type: 'torrent', text });
    }
  }

  return results;
}

/**
 * Request torrent parsing from background service worker
 */
async function requestTorrentParse(url: string): Promise<any> {
  return new Promise((resolve, reject) => {
    chrome.runtime.sendMessage(
      { action: 'downloadTorrent', url, timeout: 30000 },
      response => {
        if (chrome.runtime.lastError) {
          reject(new Error(chrome.runtime.lastError.message));
        } else {
          resolve(response);
        }
      }
    );
  });
}

// Main: scan and process
async function main(): Promise<void> {
  const links = scanPage();
  console.log(`Found ${links.length} torrent links`);

  for (const link of links) {
    if (link.type === 'magnet') {
      // Parse magnet URI locally (no network request needed)
      console.log('Magnet URI:', link.url);
      // ... extract infohash from the URL directly
      const match = link.url.match(/xt=urn:btih:([a-f0-9]{40})/i);
      if (match) {
```

```
        console.log('  InfoHash:', match[1]);
      }
    } else {
      // Download and parse .torrent file
      try {
        const result = await requestTorrentParse(link.url);
        if (result.success) {
          console.log('Torrent:', result.name);
          console.log('  InfoHash:', result.infoHash);
          console.log('  Magnet:', result.magnetUri);
        }
      } catch (error) {
        console.error('Failed to parse torrent:', error);
      }
    }
  }
}

// Run when DOM is ready
if (document.readyState === 'loading') {
  document.addEventListener('DOMContentLoaded', main);
} else {
  main();
}
```

---

14. Edge Cases and Error Handling

14.1 Known Edge Cases

Edge Case	Handling
Empty torrent file	Check `data.length === 0` before parsing
Invalid bencode	Decoder throws descriptive errors with position
Missing `info` key	Validation catches this; throw early
Missing both `length` and `files`	Validation error
Non-standard piece length	Warning (not error) - lengths like 96KB exist
Unicode filenames	Use `TextDecoder` with UTF-8 for name fields
Binary data in filenames	Store as Uint8Array, decode with care

Edge Case	Handling
Unterminated dictionaries/lists	Decoder throws with position info
Leading zeros in integers	Invalid per spec; throw error
Negative zero (`i-0e`)	Invalid per spec; throw error
Out-of-order dictionary keys	Validation warning (critical for infohash)
Private flag with DHT URLs	Valid per spec; client must respect private flag
Empty announce-list	Fall back to `announce` field
Very large .torrent files	Size validation (typically < 10MB)
Hybrid v1/v2 torrents	@ctrl/torrent-file handles both
BitTorrent v2 only	SHA-256 infohash, different structure
Passkey in announce URL	Sanitize for logging; don't leak
CORS failure on download	Service worker handles cross-origin fetch
Network timeout	Configurable timeout with AbortController
Malformed magnet URI	Validate xt parameter format

14.2 Error Recovery Strategies

```
/**
 * Robust torrent download with retry logic
 */
async function downloadTorrentRobust(
  url: string,
  options: {
    maxRetries?: number;
    timeout?: number;
    retryDelay?: number;
  } = {}
): Promise<Uint8Array> {
  const { maxRetries = 3, timeout = 30000, retryDelay = 1000 } =
        options;

  let lastError: Error | null = null;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      const controller = new AbortController();
```

```
    const timeoutId = setTimeout(() => controller.abort(),
      timeout);

    const response = await fetch(url, {
      method: 'GET',
      signal: controller.signal,
      headers: {
        'Accept': 'application/x-bittorrent,*/*',
        'User-Agent': navigator.userAgent
      }
    });

    clearTimeout(timeoutId);

    if (response.status === 404) {
      throw new Error('Torrent file not found (404)');
    }
    if (response.status === 403) {
      throw new Error('Access denied (403) - may require
      authentication');
    }
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: $
      {response.statusText}`);
    }

    const arrayBuffer = await response.arrayBuffer();
    const data = new Uint8Array(arrayBuffer);

    // Validate it looks like a torrent
    if (!TorrentParser.isTorrentFile(data)) {
      throw new Error('Downloaded file does not appear to be a
      valid torrent');
    }

    return data;

  } catch (error) {
    lastError = error instanceof Error ? error : new
      Error(String(error));

    // Don't retry on 404 or parse errors
    if (lastError.message.includes('404') ||
        lastError.message.includes('not appear to be a valid
      torrent')) {
      throw lastError;
    }

    if (attempt < maxRetries - 1) {
      await new Promise(r => setTimeout(r, retryDelay *
      (attempt + 1)));
    }
  }
```

```
  }

  throw lastError || new Error('Download failed after all
        retries');
}
```

14.3 Handling Binary Filename Encodings

Torrent files may contain filenames in various encodings. The `encoding` field at the top level hints at the encoding used:

```
function decodeTorrentString(data: Uint8Array | string,
        encoding?: string): string {
  if (typeof data === 'string') return data;

  // Try specified encoding first
  if (encoding) {
    try {
      return new TextDecoder(encoding).decode(data);
    } catch {
      // Fall through to default
    }
  }

  // Default to UTF-8
  try {
    return new TextDecoder('utf-8').decode(data);
  } catch {
    // Last resort: Latin-1 (never fails - maps bytes 1:1)
    return new TextDecoder('iso-8859-1').decode(data);
  }
}
```

---

15. Reference Library Comparison

15.1 Detailed Comparison

Library	Version	Browser-Native	MV3 Compatible	Bundle Size	TS Support	V2 Suppo
`@ctrl/torrent-file`	2.x	Yes (Uint8Array)	Yes	~8KB gzip	Full	Yes
`@substrate-system/bencode`	1.x	Yes (Uint8Array)	Yes	~5KB gzip	Full	No

Library	Version	Browser-Native	MV3 Compatible	Bundle Size	TS Support	V2 Suppo
`bencode-js`	0.x	Yes (string)	Yes	~2KB gzip	No	No
`parse-torrent`	11.x	Needs Buffer	With polyfill	~15KB+Buffer	Full	Partial
`node-bencode`	3.x	Needs Buffer	With polyfill	~10KB+Buffer	Full	No
`magnet-uri`	6.x	Yes	Yes	~3KB gzip	Full	Yes

15.2 Recommended Setup for Browser Extension (MV3)

```
// package.json dependencies
{
  "@ctrl/torrent-file": "^2.0.0",
  "magnet-uri": "^6.0.0"  // Optional, for magnet URI manipulation
}

// vite.config.ts (for building the extension)
import { defineConfig } from 'vite';
export default defineConfig({
  build: {
    target: 'es2020',
    rollupOptions: {
      input: {
        background: './src/background.ts',
        content: './src/content.ts'
      },
      output: {
        entryFileNames: '[name].js',
        chunkFileNames: '[name].js'
      }
    }
  }
});
```

Appendix A: BEP Reference

BEP	Title	URL	Status
BEP 3			Final

BEP	Title	URL	Status
	The BitTorrent Protocol Specification	https://www.bittorrent.org/beps/bep_0003.html	
BEP 9	Extension for Peers to Send Metadata Files	https://www.bittorrent.org/beps/bep_0009.html	Final
BEP 12	Multitracker Metadata Extension	https://www.bittorrent.org/beps/bep_0012.html	Final
BEP 27	Private Torrents	https://www.bittorrent.org/beps/bep_0027.html	Final
BEP 47	Padding Files and Extended File Attributes	https://www.bittorrent.org/beps/bep_0047.html	Final
BEP 52	The BitTorrent Protocol Specification v2	https://www.bittorrent.org/beps/bep_0052.html	Draft
BEP 53	Magnet URI extension	https://www.bittorrent.org/beps/bep_0053.html	Draft

Appendix B: MIME Type

The official MIME type for .torrent files:

```
application/x-bittorrent
```

This should be used in `Accept` headers when downloading and in content type validation.

Appendix C: SHA-1 via Web Crypto API

```
/**
 * Compute SHA-1 hash using Web Crypto API
 * Available in all modern browsers (Chrome, Firefox, Safari,
       Edge)
 * Note: SHA-1 is marked as "don't use in cryptographic
       applications"
 * but is still the correct algorithm for BitTorrent v1 infohash
       computation
```

```
 */
async function sha1(data: Uint8Array): Promise<Uint8Array> {
  const hashBuffer = await crypto.subtle.digest('SHA-1', data);
  return new Uint8Array(hashBuffer);
}
```

Browser Support: https://developer.mozilla.org/en-US/docs/Web/API/SubtleCrypto/digest - Chrome 37+, Firefox 34+, Safari 7+, Edge 12+ - Note: Only available in secure contexts (HTTPS) - Note: Available in Web Workers and Service Workers

---

Document Information

- **Research Date:** 2025-06-26
- **Dimension:** 07 - Torrent File Detection, Download & Bencode Parsing
- **Sources Consulted:** 30+ (BEP specifications, GitHub repos, MDN, Stack Overflow, npm docs)
- **Confidence Level:** High for core specifications (BEP 3, BEP 27), Medium-high for v2 (BEP 52, Draft status)